Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №7
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Парфенюк Д. П.
Проверил:
Кулик А. Д.

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Задание 1:

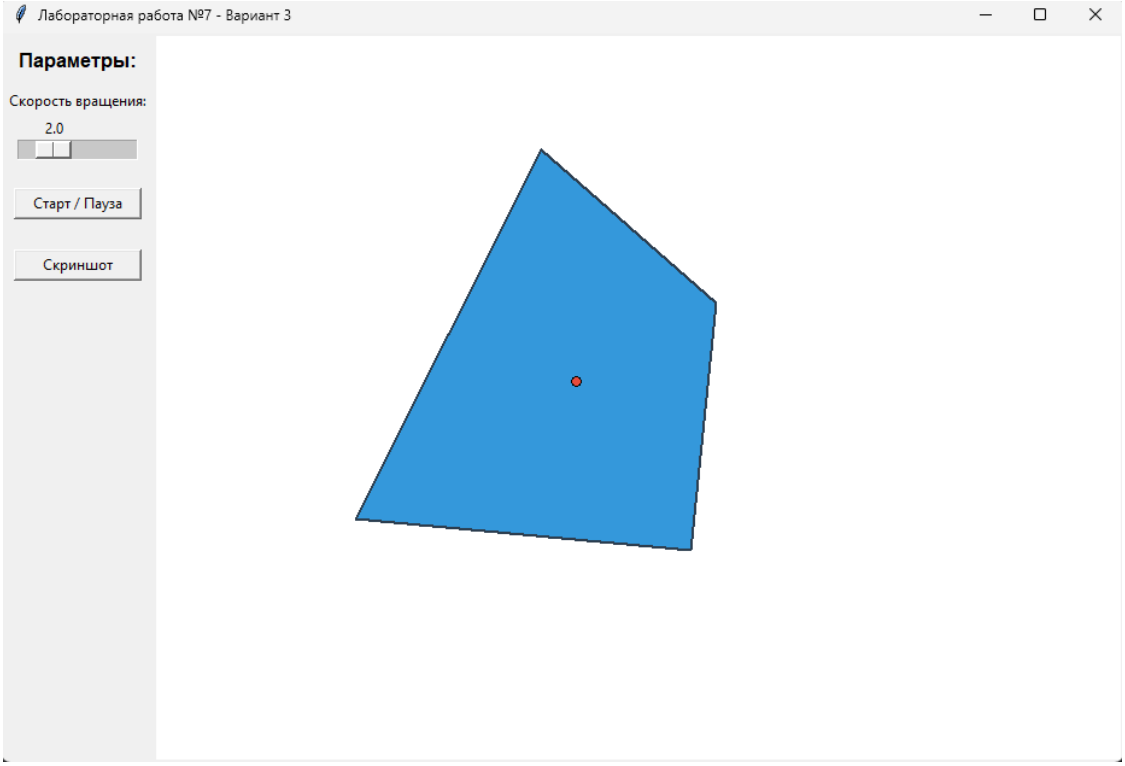
3) Изобразить четырехугольник, вращающийся в плоскости формы вокруг своего центра тяжести.

Task1lab7.py
Решение:

```
"""
Модуль для визуализации вращающегося четырехугольника.
Лабораторная работа №7, Вариант 3.
"""

import math
import time
import tkinter as tk
from tkinter import messagebox


class Point:  # pylint: disable=too-few-public-methods
    """Класс для представления точки на координатной плоскости."""

    def __init__(self, pos_x, pos_y):
        """Инициализация координат точки."""
        self.pos_x = pos_x
        self.pos_y = pos_y


class Quadrilateral:
    """Класс для управления геометрией четырехугольника."""

    def __init__(self, vertices):
        """Инициализация четырехугольника списком объектов Point."""
        self.vertices = vertices

    def get_gravity_center(self):
        """Вычисляет геометрический центр (центр тяжести) фигуры."""
        avg_x = sum(v.pos_x for v in self.vertices) / 4
        avg_y = sum(v.pos_y for v in self.vertices) / 4
        return Point(avg_x, avg_y)

    def rotate(self, angle_deg, center_pt):
        """Вращает вершины фигуры вокруг заданной точки на угол в
градусах."""
        angle_rad = math.radians(angle_deg)
        cos_val = math.cos(angle_rad)
        sin_val = math.sin(angle_rad)

        for vertex in self.vertices:
            # Смещение относительно центра вращения
            rel_x = vertex.pos_x - center_pt.pos_x
            rel_y = vertex.pos_y - center_pt.pos_y

            # Математический расчет поворота
            new_x = rel_x * cos_val - rel_y * sin_val
            new_y = rel_x * sin_val + rel_y * cos_val

            # Возвращение координат на место
            vertex.pos_x = new_x + center_pt.pos_x
```

```
            vertex.pos_y = new_y + center_pt.pos_y


class QuadApp:  # pylint: disable=too-few-public-methods
    """Класс графического приложения для анимации четырехугольника."""

    def __init__(self, master):
        """Настройка окна, интерфейса и параметров анимации."""
        self.master = master
        self.master.title("Лабораторная работа №7 - Вариант 3")

        self.is_active = True
        self.rotation_speed = tk.DoubleVar(value=2.0)

        # Создание исходного четырехугольника
        points = [
            Point(250, 200),
            Point(450, 150),
            Point(520, 420),
            Point(180, 380),
        ]
        self.quad = Quadrilateral(points)
        self.center = self.quad.get_gravity_center()

        # Настройка компонентов интерфейса
        self.canvas = tk.Canvas(self.master, width=800, height=600,
bg="white")
        self.canvas.pack(side=tk.RIGHT, expand=True, fill=tk.BOTH)

        self._setup_side_panel()
        self._animation_loop()

    def _setup_side_panel(self):
        """Создание панели управления с кнопками и слайдером."""
        panel = tk.Frame(self.master, bg="#f0f0f0", width=200)
        panel.pack(side=tk.LEFT, fill=tk.Y)

        tk.Label(panel, text="Параметры:", font=("Arial", 12,
"bold")).pack(pady=10)

        tk.Label(panel, text="Скорость вращения:").pack()
        tk.Scale(
            panel,
            from_=0,
            to=10,
            resolution=0.1,
            variable=self.rotation_speed,
            orient=tk.HORIZONTAL,
        ).pack(padx=10, fill=tk.X)

        tk.Button(panel, text="Старт / Пауза",
command=self._toggle_pause).pack(
            pady=20, fill=tk.X, padx=10
        )

        tk.Button(panel, text="Скриншот", command=self. take shot).pack(
            pady=5, fill=tk.X, padx=10
        )

    def _toggle_pause(self):
        """Переключение состояния анимации (запуск/остановка)."""
        self.is_active = not self.is_active

    def _take_shot(self):
```

```
        """Сохранение текущего вида холста в файл формата EPS."""
        file_path = f"quad_render_{int(time.time())}.eps"
        self.canvas.postscript(file=file_path)
        messagebox.showinfo("Система", f"Файл сохранен: {file_path}")

    def _animation_loop(self):
        """Главный цикл перерисовки и логики вращения."""
        if self.is_active:
            self.canvas.delete("all")

            # Вращение объекта
            self.quad.rotate(self.rotation_speed.get(), self.center)

            # Формирование списка координат для отрисовки
            flat_coords = []
            for vertex in self.quad.vertices:
                flat_coords.extend([vertex.pos_x, vertex.pos_y])

            # Рисование фигуры
            self.canvas.create_polygon(
                flat_coords, fill="#3498db", outline="#2c3e50", width=2
            )

            # Визуализация центра тяжести
            radius = 4
            self.canvas.create_oval(
                self.center.pos_x - radius,
                self.center.pos_y - radius,
                self.center.pos_x + radius,
                self.center.pos_y + radius,
                fill="#e74c3c",
            )

        # Запланировать следующий кадр через 20 мс
        self.master.after(20, self._animation_loop)


if __name__ == "__main__":
    root = tk.Tk()
    app = QuadApp(root)
    root.mainloop()
```



Задание 2:

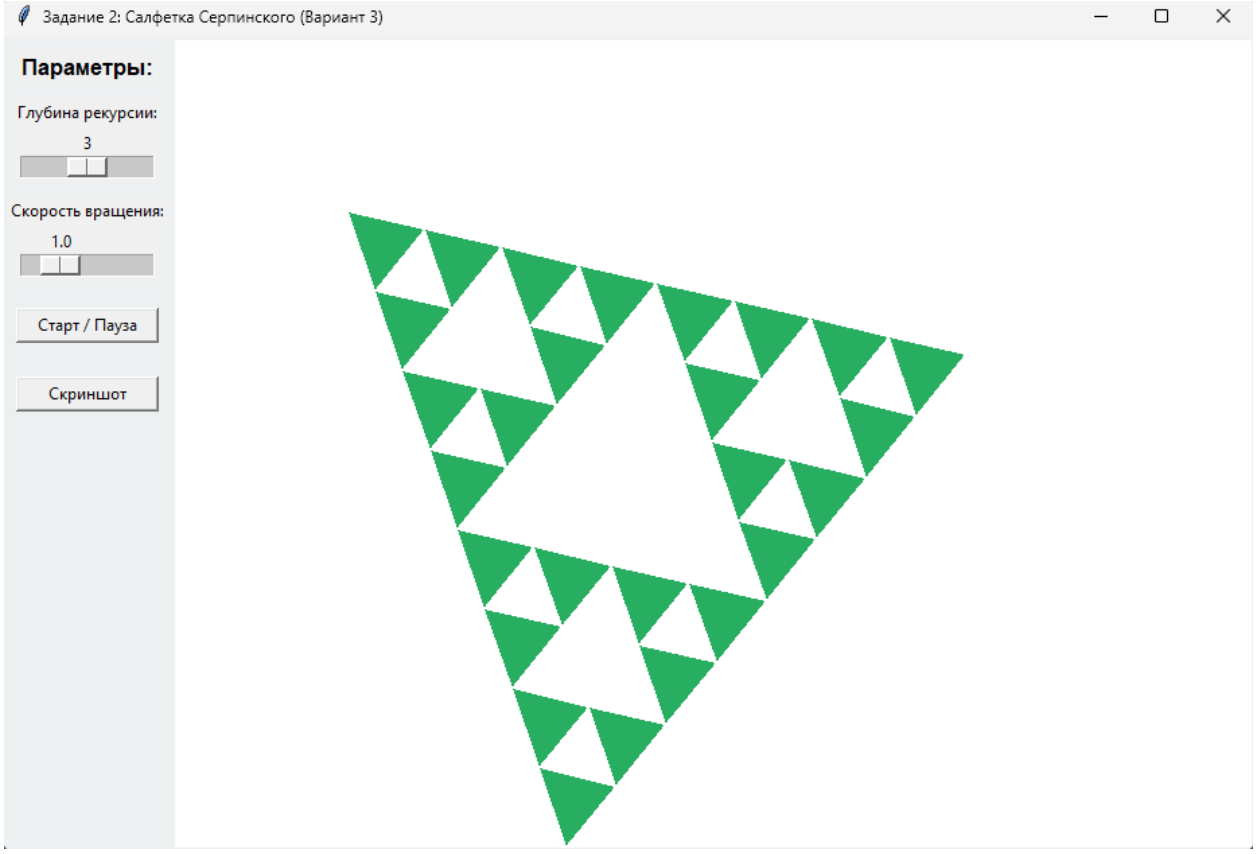
3) Треугольная салфетка Серпинского

Task2lab7.py
Решение:

```
"""
Модуль для построения фрактала 'Треугольная салфетка Серпинского'.
Лабораторная работа №7, Вариант 3.
Реализовано динамическое вращение и изменение глубины рекурсии.
"""

import math
import time
import tkinter as tk
from tkinter import messagebox


class Point:  # pylint: disable=too-few-public-methods
    """Класс для представления точки на координатной плоскости."""

    def __init__(self, pos_x, pos_y):
        """Инициализация координат точки."""
        self.pos_x = pos_x
        self.pos_y = pos_y


class SierpinskiApp:  # pylint: disable=too-few-public-methods
    """Класс графического приложения для визуализации фрактала."""

    def __init__(self, master):
        """Настройка окна, интерфейса и начальных параметров фрактала."""
        self.master = master
        self.master.title("Задание 2: Салфетка Серпинского (Вариант 3)")
```

```
        self.is_active = True
        self.rotation_angle = 0.0
        self.rotation_speed = tk.DoubleVar(value=1.0)
        self.recursion_depth = tk.IntVar(value=3)

        # Центр экрана для вращения
        self.screen_center = Point(400, 300)

        # Настройка компонентов
        self.canvas = tk.Canvas(self.master, width=800, height=600,
bg="white")
        self.canvas.pack(side=tk.RIGHT, expand=True, fill=tk.BOTH)

        self._setup_side_panel()
        self._animation_loop()

    def _setup_side_panel(self):
        """Создание боковой панели управления."""
        panel = tk.Frame(self.master, bg="#ecf0f1", width=200)
        panel.pack(side=tk.LEFT, fill=tk.Y)

        tk.Label(panel, text="Параметры:", font=("Arial", 12,
"bold")).pack(pady=10)

        tk.Label(panel, text="Глубина рекурсии:").pack()
        tk.Scale(
            panel,
            from_=0,
            to=6,
            variable=self.recursion_depth,
            orient=tk.HORIZONTAL,
        ).pack(padx=10, fill=tk.X)

        tk.Label(panel, text="Скорость вращения:").pack(pady=(10, 0))
        tk.Scale(
            panel,
            from_=0,
            to=5,
            resolution=0.1,
            variable=self.rotation_speed,
            orient=tk.HORIZONTAL,
        ).pack(padx=10, fill=tk.X)

        tk.Button(panel, text="Старт / Пауза",
command=self._toggle_pause).pack(
            pady=20, fill=tk.X, padx=10
        )

        tk.Button(panel, text="Скриншот", command=self._take_shot).pack(
            pady=5, fill=tk.X, padx=10
        )

    def _toggle_pause(self):
        """Включение и выключение анимации."""
        self.is_active = not self.is_active

    def _take_shot(self):
        """Сохранение изображения в формате EPS."""
        file_name = f"sierpinski_{int(time.time())}.eps"
        self.canvas.postscript(file=file_name)
        messagebox.showinfo("Система", f"Фрактал сохранен: {file_name}")

    def _rotate_point(self, point, angle_deg):
        """Математический расчет поворота точки вокруг центра экрана."""
```

```
        angle_rad = math.radians(angle_deg)
        rel_x = point.pos_x - self.screen_center.pos_x
        rel_y = point.pos_y - self.screen_center.pos_y

        new_x = rel_x * math.cos(angle_rad) - rel_y * math.sin(angle_rad)
        new_y = rel_x * math.sin(angle_rad) + rel_y * math.cos(angle_rad)

        return Point(new_x + self.screen_center.pos_x, new_y +
self.screen_center.pos_y)

    def _get_midpoint(self, p1, p2):
        """Находит середину отрезка между двумя точками."""
        return Point((p1.pos_x + p2.pos_x) / 2, (p1.pos_y + p2.pos_y) / 2)

    def _draw_recursive(self, p1, p2, p3, depth):
        """Рекурсивная функция отрисовки треугольников Серпинского."""
        if depth == 0:
            self.canvas.create_polygon(
                [p1.pos_x, p1.pos_y, p2.pos_x, p2.pos_y, p3.pos_x, p3.pos_y],
                fill="#27ae60",
                outline="white",
            )
        else:
            mid1 = self._get_midpoint(p1, p2)
            mid2 = self._get_midpoint(p2, p3)
            mid3 = self._get_midpoint(p1, p3)

            self._draw_recursive(p1, mid1, mid3, depth - 1)
            self._draw_recursive(mid1, p2, mid2, depth - 1)
            self._draw_recursive(mid3, mid2, p3, depth - 1)

    def _animation_loop(self):
        """Главный цикл анимации и перерисовки."""
        if self.is_active:
            self.canvas.delete("all")
            self.rotation_angle += self.rotation_speed.get()

            # Базовые точки большого треугольника
            p1 = Point(400, 100)
            p2 = Point(150, 500)
            p3 = Point(650, 500)

            # Поворот базовых точек
            rp1 = self._rotate_point(p1, self.rotation_angle)
            rp2 = self._rotate_point(p2, self.rotation_angle)
            rp3 = self._rotate_point(p3, self.rotation_angle)

            # Отрисовка фрактала
            self._draw_recursive(rp1, rp2, rp3, self.recursion_depth.get())

        self.master.after(30, self._animation_loop)


if __name__ == "__main__":
    root = tk.Tk()
    app = SierpinskiApp(root)
    root.mainloop()
```



Вывод: освоил возможности языка программирования Python в разработке оконных приложений.